LIS-VLA: Latent Imagination Search for Vision-Language-Action Models

Method and Implementation Report

Yike Wang
University of Wollongong
yw061@uowmail.edu.au

Abstract

Vision-language-action (VLA) models usually map an observation and an instruction to an action in a single forward pass, committing to one sample without checking where it leads. We describe *LIS-VLA* (Latent Imagination Search VLA), which turns this feed-forward mapping into an internal propose–imagine–evaluate loop that runs entirely in a learned latent space. From one forward pass of a unified VLA backbone, a flow-matching proposer draws N candidate action chunks; a latent world model predicts the latent state each chunk leads to; a language-guided evaluator scores every imagined future against the instruction; and the highest-scoring chunk, minus a smoothness cost, is executed. No pixels are generated during search and the backbone is not re-run per candidate. All System-2 components are trained from demonstrations only, using a JEPA-style latent prediction target, contrastive language grounding, and ranking against counterfactual chunks. We instantiate LIS-VLA on RynnVLA-002, keeping its checkpoints loadable, and release the implementation. This report documents the formulation, the integration details, and verification of the implementation, including a controlled toy experiment in which imagination search selects chunks with the instructed effect from uniformly random candidates well above chance. **We do not yet report robot-benchmark results**; the evaluation protocol for LIBERO is specified and left to a future version of this report.

1 Introduction

Vision-language-action models such as RT-2 [5], OpenVLA [17] and π_0 [4] have made it practical to drive manipulators from images and free-form instructions. Architecturally, most of them are reactive: perception and language are encoded once and an action (or action chunk [26]) is decoded once. When the decoded action is subtly wrong—a grasp that is a few centimetres off, a push in the wrong direction—nothing in the forward pass notices.

Two lines of work suggest ways around this. First, world models learned for control, from PlaNet [12] and DreamerV3 [13] to TD-MPC2 [14], DINO-WM [27] and V-JEPA 2 [2], show that planning by *imagining* the consequences of candidate actions in a learned latent space is effective and much cheaper than predicting pixels. Second, test-time sampling and verification for VLAs, e.g. RoboMonkey [18] and value-guided steering [23], shows that re-ranking several sampled actions with a learned scorer improves a fixed policy. Meanwhile, unified models such as WorldVLA [7] and RynnVLA-002 [6] already train one autoregressive backbone both to act and to predict future frames, and report that the two abilities enhance each other. In RynnVLA-002's released evaluation code, however, actions are produced by the action head alone; the world model is not consulted when choosing an action. Its predictions are also made in image-token space, which would be too slow to run for many candidates per control step.

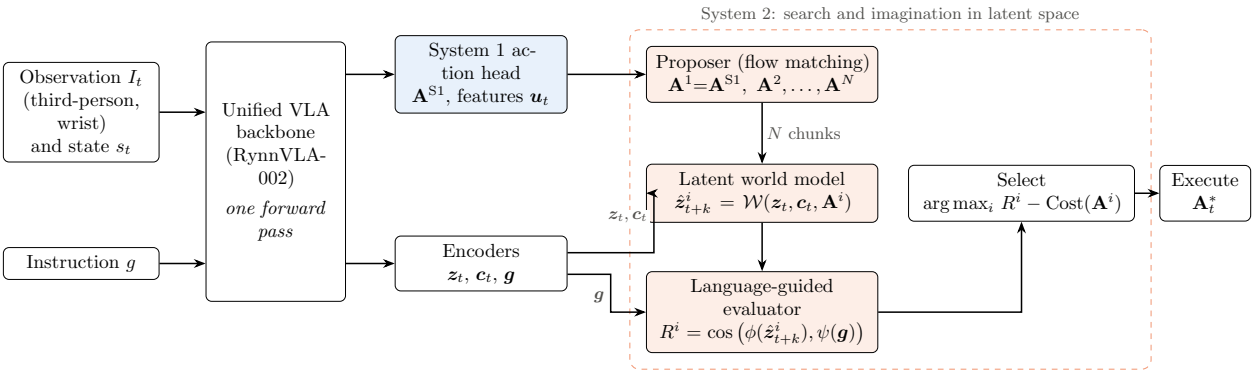


Figure 1: LIS-VLA. A single backbone pass feeds the original action head (System 1) and the System-2 encoders. The proposer samples N chunks (the first candidate is the System-1 chunk), the latent world model imagines the state each leads to, the evaluator scores the imagined futures against the instruction, and the best chunk is executed. Search never generates pixels or re-runs the backbone.

LIS-VLA combines these ideas in a unified VLA. In the spirit of the fast/slow distinction of Kahneman [16], the original action head of the backbone acts as a fast *System 1*, and we add a lightweight *System 2* that searches over candidate action chunks by imagination (Figure 1). Concretely, our contributions are:

1. A formulation of action selection as language-conditioned search in latent space,

$$\mathbf{A}^*_t = \arg\max_{\mathbf{A}} \ \mathrm{Sim}\big(\phi(\hat{\boldsymbol{z}}_{t+k}), \psi(g)\big) - \mathrm{Cost}(\mathbf{A}),$$

   where $\hat{\boldsymbol{z}}_{t+k}$ is predicted by a latent world model from the current latent state and a candidate chunk (Section 4).

2. An instantiation on RynnVLA-002 in which all System-2 modules read from *one* backbone forward pass, add about 50M parameters to a 7B model, and leave released checkpoints loadable (Section 5).

3. A set of training objectives that need only demonstrations: flow matching for the proposer, a stop-gradient latent prediction target with anti-collapse terms for the world model, and contrastive, progress and counterfactual-ranking losses for the evaluator, together with a data format in which a single token sequence supervises all of them (Section 4.7).

4. An open-source implementation with unit tests and a controlled toy experiment that verifies the search mechanism end to end (Section 6), plus a concrete LIBERO [20] evaluation protocol (Section 7).

This document is deliberately scoped as a method and implementation report: the large-scale experiments needed to establish whether latent imagination search improves task success have not yet been run, and we make no claims about robot performance.

2 Related Work

Vision-language-action models. RT-2 [5] and OpenVLA [17] discretise actions into language-model tokens; π_0 [4] attaches a flow-matching action expert, and Diffusion Policy [9] and ACT [26]

popularised generative heads and action chunking. These models are generative and can be sampled, but are deployed as single-shot policies. LIS-VLA keeps such a policy as System 1 and uses a generative proposer only to supply candidates for search.

World models inside VLAs. UniPi [10] plans by generating a video and inferring actions from it. WorldVLA [7] and RynnVLA-002 [6] train a single autoregressive model, built on a Chameleon-style [8] early-fusion backbone, both as a policy and as an action-conditioned image predictor, and report that the two objectives help each other. We build directly on RynnVLA-002 but use a separate, small *latent* dynamics head for inference-time search, because predicting hundreds of image tokens per candidate is incompatible with evaluating many candidates per control step.

Latent world models and planning. PlaNet [12] introduced planning with the cross-entropy method [24] in a learned latent space; DreamerV3 [13] and TD-MPC2 [14] scaled latent models for control. DINO-WM [27] and V-JEPA 2 [2] plan zero-shot towards goal *images* with dynamics learned on top of self-supervised visual features; our latent target follows the joint-embedding predictive idea of I-JEPA [1]. LIS-VLA differs in that the goal is a *language instruction*, the latent space is derived from the VLA's own token embeddings, and the dynamics model shares its conditioning with the policy.

Test-time sampling and verification for robot policies. RoboMonkey [18] samples actions from a VLA, perturbs and aggregates them into a proposal set, and selects one with a VLM-based verifier trained on synthetic data; V-GPS [23] re-ranks samples of a generalist policy with a value function learned by offline RL. LIS-VLA also samples and selects, but its scorer evaluates the *predicted consequence* of an action chunk, $f(\hat{\boldsymbol{z}}_{t+k}, g)$, rather than the action itself, and the scorer is trained jointly with the world model from the same demonstrations.

3 Background: RynnVLA-002

RynnVLA-002 [6] serialises every modality into one token stream for a Chameleon-style autoregressive transformer (7B parameters in the released models, with the Lumina-mGPT [21] image tokenizer). An image is a block of VQ tokens delimited by start/end markers; each 7-D action is normalised to $[-1, 1]$, discretised into 256 bins and wrapped in action-start/end markers; the proprioceptive state is encoded analogously. A VLA training sample has the form

$$\underbrace{\text{instruction}}_{g}\ \underbrace{\langle\text{state}\rangle}_{s_t}\ \underbrace{\langle\text{img}\rangle \times (h \cdot V)}_{\text{history } h,\ V \text{ views}}\ \underbrace{\langle\text{act}\rangle \times H}_{\text{chunk}}$$

and world-model samples ask the model to generate the next frame given images and an action. Besides next-token prediction, RynnVLA-002 has a continuous action head: $H \cdot 7$ learned query tokens are appended to the backbone's last-layer hidden states of the prompt, processed by a small transformer encoder (width 1024 for the 7B model), and regressed to the chunk $\mathbf{A} \in [-1, 1]^{H \times 7}$ with an L_1 loss. Within a chunk, a custom attention mask prevents action blocks from attending to each other. At inference the prompt is extended by a single action-start token and one backbone pass yields the hidden states from which the head predicts the whole chunk. We call this deterministic chunk $\mathbf{A}^{\mathrm{S1}}$.

4 Method

4.1 Overview

Given the backbone's last-layer hidden states $\boldsymbol{h}$ for the prompt (instruction, state and images up to time t), LIS-VLA computes

$$\boldsymbol{z}_t = E_{\text{obs}}(I_t), \qquad \boldsymbol{c}_t = E_{\text{ctx}}(\boldsymbol{h}_{<a}), \qquad \boldsymbol{g} = E_{\text{lang}}(\boldsymbol{h}_{\text{instr}}), \tag{1}$$

$$\{\mathbf{A}^i\}_{i=1}^{N} \sim p_\theta(\mathbf{A} \mid \boldsymbol{u}_t, \boldsymbol{c}_t), \qquad \mathbf{A}^1 := \mathbf{A}^{\text{S1}}, \tag{2}$$

$$\hat{\boldsymbol{z}}^i_{t+k} = \mathcal{W}(\boldsymbol{z}_t, \boldsymbol{c}_t, \mathbf{A}^i), \tag{3}$$

$$\mathbf{A}^*_t = \arg\max_i \underbrace{\cos\big(\phi(\hat{\boldsymbol{z}}^i_{t+k}),\, \psi(\boldsymbol{g})\big)}_{\text{Sim}} - \text{Cost}(\mathbf{A}^i), \tag{4}$$

where $\boldsymbol{h}_{<a}$ are the hidden states before the action-start token, $\boldsymbol{h}_{\text{instr}}$ those of the instruction tokens, and $\boldsymbol{u}_t$ the action-query features of the System-1 head. The horizon k equals the chunk length H, so one world-model call imagines the effect of one whole chunk. Because the world model is deterministic, the expectation over imagined futures in a stochastic formulation reduces to a single prediction.

4.2 Latent state, context and goal

Observation latents $\boldsymbol{z}_t$. We do not use the backbone's *hidden* states at image positions as the latent state, because in a causal model they also encode everything that precedes them and, at future-frame positions during training, the actions themselves. Instead, E_{obs} reads the backbone's *input embeddings* of the VQ tokens of the V current views, adds learned within-view position and view embeddings, and pools them with Q learned queries in a Perceiver-style [15] cross-attention block followed by one self-attention layer, giving $\boldsymbol{z}_t \in \mathbb{R}^{Q\times d}$ (Q=16, d=512). Since $\boldsymbol{z}_t$ depends only on the observation, the same encoder applied to the real future frame yields the world-model target $\boldsymbol{z}_{t+k}$.

Context and goal. E_{ctx} and E_{lang} are attention poolers over the backbone hidden states, producing K=8 context tokens $\boldsymbol{c}_t$ and a single goal vector $\boldsymbol{g}$. Because the instruction precedes all images in the sequence, causal attention guarantees that $\boldsymbol{g}$ depends on the instruction only.

4.3 Proposer

The proposer is a rectified-flow [19, 22] model over chunks. Its velocity network is a small transformer over H action tokens and the context tokens $\boldsymbol{c}_t$; each action token is conditioned on the matching System-1 query features $\boldsymbol{u}_t$ and a time embedding. Sampling integrates $\dot{\boldsymbol{x}} = v_\theta(\boldsymbol{x}, \tau)$ from $\boldsymbol{x}_0 \sim \mathcal{N}(0, \sigma^2 I)$ with 10 Euler steps for N chains in parallel and clips to $[-1, 1]$. Candidate #1 is always $\mathbf{A}^{\text{S1}}$, so with Equation (4) the selected chunk can never score lower than the System-1 chunk under the model's own criterion.

4.4 Latent world model

$\mathcal{W}$ is a transformer over the concatenation of $\boldsymbol{z}_t$, $\boldsymbol{c}_t$ and H embedded actions (with per-step position embeddings), whose outputs at the Q latent positions are mapped back through a residual linear layer: $\hat{\boldsymbol{z}}_{t+k} = \boldsymbol{z}_t + W_o\,\text{TF}([\boldsymbol{z}_t; \boldsymbol{c}_t; \mathbf{A}])_{1:Q}$. W_o is zero-initialised, so an untrained model predicts that nothing changes, which is a safe default for search.

Algorithm 1 LIS-VLA action selection at time t

Require: prompt tokens (instruction g, state, images), candidates N, CEM rounds C, elites n_e

$\boldsymbol{h} \leftarrow$ backbone hidden states for the prompt and the action-start token ▷ one pass

$\mathbf{A}^{\mathrm{S1}}, \boldsymbol{u}_t \leftarrow$ System-1 action head$(\boldsymbol{h})$

$\boldsymbol{z}_t, \boldsymbol{c}_t, \boldsymbol{g} \leftarrow E_{\mathrm{obs}}$(image tokens), $E_{\mathrm{ctx}}(\boldsymbol{h}_{<a})$, $E_{\mathrm{lang}}(\boldsymbol{h}_{\mathrm{instr}})$

$\{\mathbf{A}^i\}_{i=1}^{N} \leftarrow$ proposer samples; $\mathbf{A}^1 \leftarrow \mathbf{A}^{\mathrm{S1}}$

for $c = 0, \ldots, C$ **do**

  $\hat{\boldsymbol{z}}^i \leftarrow \mathcal{W}(\boldsymbol{z}_t, \boldsymbol{c}_t, \mathbf{A}^i)$ for all i ▷ batched

  $S^i \leftarrow \cos(\phi(\hat{\boldsymbol{z}}^i), \psi(\boldsymbol{g})) - \mathrm{Cost}(\mathbf{A}^i)$

  if $c < C$ **then**

    $\mathcal{E} \leftarrow$ top-n_e candidates by S; $\mu, \sigma \leftarrow$ mean, std of $\mathcal{E}$

    $\{\mathbf{A}^i\} \leftarrow \mathcal{E} \cup \{\mathrm{clip}(\mu + \sigma \odot \epsilon_j)\}_{j=1}^{N-n_e}$

  end if

end for

return $\mathbf{A}_t^* = \mathbf{A}^{\arg\max_i S^i}$ ▷ executed as a chunk

4.5 Evaluator and cost

The evaluator pools $\hat{\boldsymbol{z}}_{t+k}$ to one vector and computes $R = \cos(\phi(\hat{\boldsymbol{z}}_{t+k}), \psi(\boldsymbol{g}))$ with two MLP heads ϕ, ψ. The cost penalises jerky arm motion and, optionally, deviation from System 1:

$$\mathrm{Cost}(\mathbf{A}) = \frac{\lambda_s}{H-1} \sum_{i=1}^{H-1} \left\| \boldsymbol{a}_{i+1}^{\mathrm{arm}} - \boldsymbol{a}_i^{\mathrm{arm}} \right\|^2 + \frac{\lambda_p}{H} \left\| \mathbf{A} - \mathbf{A}^{\mathrm{S1}} \right\|_F^2, \tag{5}$$

where $\boldsymbol{a}^{\mathrm{arm}}$ excludes the gripper dimension (gripper commands are meant to switch). Defaults are $\lambda_s = 0.05$, $\lambda_p = 0$.

4.6 Search

Algorithm 1 summarises inference. All candidates are processed as one batch by the System-2 heads; optional cross-entropy-method (CEM) rounds [12, 24] refit a diagonal Gaussian to the top-n_e candidates and resample the rest, always retaining the elites.

4.7 Training from demonstrations

Data. We extend each VLA sample with the frames that are actually observed after the chunk has been executed:

$$\text{instruction } \langle\text{state}\rangle\ \langle\text{img}\rangle \times (h \cdot V)\ \ \langle\text{act}\rangle \times H\ \ \underbrace{\langle\text{img}\rangle \times V}_{\text{frames at } t+H}\ .$$

One such sequence supervises the System-1 head, the proposer, the world model and the evaluator, and its next-token loss additionally trains the backbone to generate the frame at $t + H$, in the spirit of RynnVLA-002's action-world-model data. Samples whose trajectory ends before $t + H$ are dropped; plain VLA and world-model samples can be mixed in.

Losses. Let $\mathrm{sg}[\cdot]$ denote stop-gradient, $\boldsymbol{z}^+ = \mathrm{sg}[E_{\mathrm{obs}}(I_{t+H})]$ the latent of the real future, $\hat{\boldsymbol{z}} = \mathcal{W}(\boldsymbol{z}_t, \boldsymbol{c}_t, \mathbf{A}^{\mathrm{demo}})$ the predicted one, $\bar{\boldsymbol{z}}$ the mean over latent tokens, and $R(\boldsymbol{z}, \boldsymbol{g}) = \cos(\phi(\boldsymbol{z}), \psi(\boldsymbol{g}))$. In addition to RynnVLA-002's next-token and L_1 losses we minimise $\sum_j w_j \mathcal{L}_j$ with:

- $\mathcal{L}_{\text{fm}} = \mathbb{E}\,\|v_\theta(\boldsymbol{x}_\tau, \tau) - (\mathbf{A} - \boldsymbol{x}_0)\|^2$ with $\boldsymbol{x}_\tau = (1-\tau)\boldsymbol{x}_0 + \tau\mathbf{A}$ and logit-normal τ [11] (w=1.0)
- $\mathcal{L}_{\text{wm}} = \frac{1}{Q}\sum_q \big(1 - \cos(\hat{\boldsymbol{z}}_q, \boldsymbol{z}_q^+)\big)$, a JEPA-style [1] latent target (w=1.0)
- $\mathcal{L}_{\text{nce}}$: InfoNCE [25] over the batch between $\bar{\hat{\boldsymbol{z}}}$ and $\bar{\boldsymbol{z}}^+$, so each prediction must identify its own future (w=0.1)
- $\mathcal{L}_{\text{var}}$: the VICReg [3] variance hinge on $\bar{\boldsymbol{z}}_t$ (w=0.1)
- $\mathcal{L}_{\text{lang}}$: multi-positive InfoNCE between $\phi(\boldsymbol{z}^+)$, $\phi(\text{sg}[\hat{\boldsymbol{z}}])$ and all goals $\psi(\boldsymbol{g}_j)$ in the batch, with samples sharing an instruction treated as positives (w=0.1)
- $\mathcal{L}_{\text{prog}} = \text{softplus}\big(\kappa\,[R(\text{sg}[\boldsymbol{z}_t], \boldsymbol{g}) - R(\boldsymbol{z}^+, \boldsymbol{g})]\big)$: the reached state should match the instruction better than the current one (w=0.05)
- $\mathcal{L}_{\text{rank}} = \mathbb{E}_{\tilde{\mathbf{A}}}\,\text{softplus}\big(\kappa\,[R(\mathcal{W}(\boldsymbol{z}_t, \boldsymbol{c}_t, \tilde{\mathbf{A}}), \boldsymbol{g}) - R(\text{sg}[\hat{\boldsymbol{z}}], \boldsymbol{g})]\big)$, where the counterfactual chunks $\tilde{\mathbf{A}}$ are the demonstration chunk with Gaussian noise (σ=0.3), reversed in time, or taken from another sample in the batch (w=0.1)

κ is the evaluator's learned inverse temperature. $\mathcal{L}_{\text{rank}}$ updates only the evaluator (and goal encoder): the world model is trained exclusively by prediction losses, so it cannot reduce the ranking loss by distorting its predictions. The two InfoNCE terms and the variance term prevent the trivial solution in which E_{obs} maps all observations to a constant. By default the System-2 losses do not back-propagate into the backbone.

5 Implementation

Integration. LIS-VLA subclasses RynnVLA-002's model, keeps its training loop unchanged (the new losses are returned through the model's auxiliary-loss dictionary), and adds training and LIBERO evaluation entry points, a data generator for the format above and shell scripts for all four LIBERO suites. Three training scopes are exposed: `lis_only` (backbone and action head frozen; the cheapest way to add System 2 to a released checkpoint), `lis_and_head`, and `all`.

Two corrections. (i) *Attention mask.* RynnVLA-002 decides whether action blocks may attend to each other by comparing them with the position of the *last* image in the sequence. In our training sequences future frames follow the chunk, so this rule would let the chunk's action blocks see each other during training but not at inference. We use the last image *before the first action block* instead; for all of RynnVLA-002's own sample layouts and for inference prompts the resulting mask is identical to the original (checked by a unit test). (ii) *Padding in the action head.* The original head lets action queries attend to zero padding when prompts in a batch have different lengths; we mask padded positions. Parameter names and shapes are unchanged, so released checkpoints load directly, and for batch size one the head is numerically identical to the original (checked by a unit test).

Size. Table 1 lists the System-2 parameter count at the dimensions of the 7B backbone: about 50M parameters, roughly 0.7% of the backbone. Search adds no backbone passes and generates no image tokens.

Table 1: System-2 parameters at 7B-backbone dimensions (hidden size 4096, action-head width 1024, H=5, latent width d=512).

Module	Parameters (M)
Observation encoder E_{obs}	9.0
Context encoder E_{ctx}	5.3
Goal encoder E_{lang}	5.3
Flow-matching proposer	13.7
Latent world model $\mathcal{W}$	12.9
Evaluator (ϕ, ψ)	3.9
Total	50.1

6 Verification

Because robot-scale experiments are not yet available, we verify (a) that the implementation is correct and compatible with RynnVLA-002 and (b) that the search mechanism behaves as intended in a controlled setting. Neither says anything about manipulation performance.

Unit tests. With a tiny Chameleon configuration (2 layers, width 64) and the exact RynnVLA-002 token layout, seven tests check: sequence parsing; equality of our attention mask with the original on VLA samples and inference prompts, and the corrected masking on LIS samples; that a training step yields finite values for all seven System-2 losses and non-zero gradients in every System-2 module; that the proposer, world-model and language losses decrease when the System-2 heads are optimised; end-to-end `generate` with search, with CEM and with search disabled; state-dict compatibility and numerical equality of the action head with the original; and loading a checkpoint that lacks System-2 weights. All tests pass with the library versions pinned by RynnVLA-002 (PyTorch 2.2.0, Transformers 4.43.0).

Controlled toy experiment. We construct synthetic sequences in the RynnVLA-002 format whose dynamics are known. Four instructions are split into two goals, "marker A" and "marker B". Images are random VQ tokens; the frame after a chunk equals the current frame with the first six tokens of each view replaced by marker A if the mean of action dimension 0 over the chunk is positive and by marker B otherwise. Demonstrations always move towards the instructed marker; the other action dimensions are random. Only the System-2 heads are trained (the tiny backbone is random and frozen, i.e. the `lis_only` scope), on 256 training sequences for 400 steps with batch size 16. We evaluate on 200 held-out sequences with new random images and report (i) the cosine similarity between predicted and true future latents, against the baseline of copying $\boldsymbol{z}_t$; (ii) *action sensitivity*: how often the prediction from the demonstration chunk is closer to the true future than the prediction from the same chunk with dimension 0 negated; and (iii) *search accuracy*: we draw N chunks *uniformly at random* (no proposer, no System-1 candidate, no cost), select by $\arg\max_i R^i$, and count how often the selected chunk produces the instructed marker. Each random candidate is correct with probability 0.5, so this isolates the world model and evaluator.

Table 2 and Figure 2 summarise the results. After training, the world model is action-sensitive in 75.8% of held-out cases, and selecting among random chunks with the evaluator's score raises the fraction of chunks with the instructed effect from 47.7% at N=1 (chance) to 99.7% at N=32. The untrained System 2 stays near chance, as expected from its identity dynamics. The trained proposer alone also learns to produce the instructed effect, so in this toy task search is not *needed*;

Table 2: Controlled toy experiment on 200 held-out sequences (mean ± s.d. over 3 seeds). "Before" is the untrained System 2 (zero-initialised world model, i.e. identity dynamics); "after" is after training. Cosine similarities are computed between flattened latent sets; note that E_{obs} is itself trained, so the copy baseline changes too.

Metric	Before	After
$\cos(\hat{\boldsymbol{z}}_{t+H}, \boldsymbol{z}_{t+H})$	0.729 ± 0.139	0.952 ± 0.011
$\cos(\boldsymbol{z}_t, \boldsymbol{z}_{t+H})$ (copy baseline)	0.729 ± 0.139	0.890 ± 0.068
Action sensitivity (%)	50.0 ± 0.0	75.8 ± 11.8
Proposer samples with instructed effect (%)	50.4 ± 2.4	99.2 ± 0.5

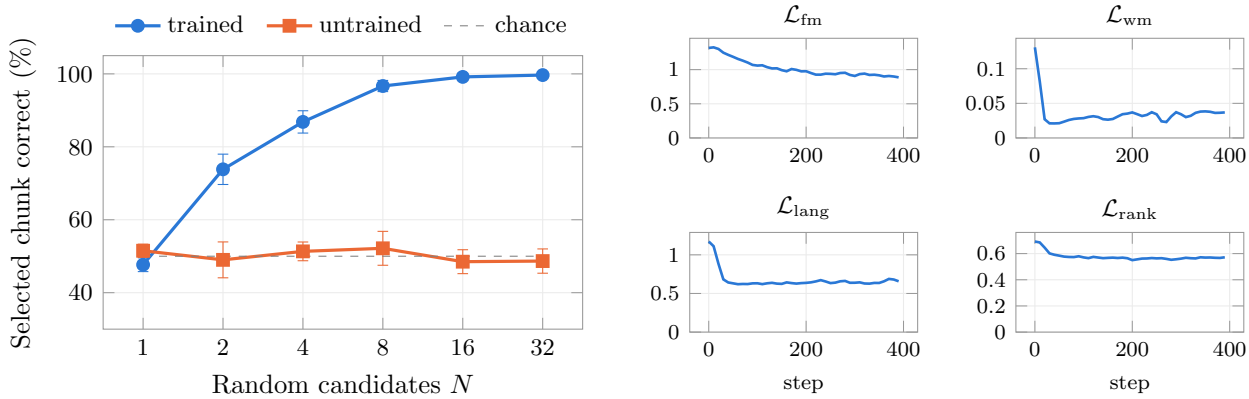


Figure 2: Toy experiment. *Left:* imagination search over N uniformly random chunks; a chunk is correct if it produces the instructed marker (each random chunk is correct with probability 0.5). Error bars: s.d. over 3 seeds, 200 held-out sequences each. *Right:* training losses (mean over seeds, 20-step moving average).

the experiment is designed only to show that the world model and evaluator implement a working selection signal when the dynamics are learnable from the latent state.

Latency. Measured for the System-2 heads alone at 7B-backbone dimensions (50.1M parameters) on CPU (x86_64), 2 threads, fp32 with PyTorch 2.14.1; the backbone pass is shared with System 1 and excluded. Median of 5 runs, in milliseconds per control step:

N	1	4	8	16	32	64	16+CEM2
ms	108	162	206	381	626	1232	506

These CPU numbers only bound the cost of search from above; GPU latency, which is what matters for deployment, has not been measured.

7 Planned Evaluation on LIBERO

The next version of this report will evaluate on the four LIBERO suites [20] (Spatial, Object, Goal, Long) with RynnVLA-002's released checkpoints and evaluation settings (two history frames, third-person and wrist views, state input, chunk length 5 for Goal/Object and 10 for Spatial/Long, 50 episodes per task). The planned comparisons, each on the same checkpoint where possible, are:

1. System 1 only vs. LIS-VLA, with $N \in \{4, 8, 16, 32\}$ and with/without CEM; success rate and wall-clock control latency on GPU;
2. evaluator ablations removing $\mathcal{L}_{\text{rank}}$, $\mathcal{L}_{\text{prog}}$ or $\mathcal{L}_{\text{lang}}$;
3. latent-space search vs. pixel-space search that scores frames generated by RynnVLA-002's own world model, at matched candidate budgets;
4. training scopes `lis_only` vs. `all`;
5. diagnostics from the search logs: how often the System-1 chunk is overridden, and the spread of scores among candidates.

8 Limitations

- **No robot results.** Whether imagination search improves task success on LIBERO or real robots is untested. The toy experiment only shows that the mechanism can work when the dynamics are learnable and the goal is identifiable from the latent state.
- **Demonstration-only evaluator.** The evaluator is trained from expert data and counterfactual perturbations of it. It therefore encodes a preference for futures that resemble expert progress, not a calibrated success probability, and may be unreliable for out-of-distribution candidates; search can exploit its errors.
- **One-chunk, deterministic imagination.** The world model predicts a single latent outcome H steps ahead and does not represent uncertainty or roll out multiple chunks.
- **Latent space.** Latents are pooled from VQ-token embeddings, which may discard fine geometric detail that matters for contact-rich steps.
- **Latency.** Search latency has only been measured on CPU.

9 Conclusion

LIS-VLA adds a small, latent-space System 2 to a unified VLA: propose with flow matching, imagine with a latent world model, judge against the instruction, and act. It reuses a single backbone pass, trains from demonstrations only and remains compatible with RynnVLA-002 checkpoints. The implementation and a controlled sanity check are complete; establishing whether the approach improves manipulation is the subject of the planned evaluation.

Code Availability

Code, training and evaluation scripts, and the toy experiment: https://github.com/wangyk10086/LIS-VLA.

Acknowledgements

LIS-VLA builds on the code and models released by the authors of RynnVLA-002 [6]. The implementation and an initial draft of this report were prepared with the assistance of Claude (Anthropic), an AI system; the author reviewed the code and text and takes full responsibility for the content.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023.

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. *arXiv preprint arXiv:2105.04906*, 2021.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[6] Jun Cen, Siteng Huang, Yuqian Yuan, Kehan Li, Hangjie Yuan, Chaohui Yu, Bohan Hou, Yuming Jiang, Jiayan Guo, Xin Li, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. RynnVLA-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*, 2025.

[7] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.

[8] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.

[9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.

[10] Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *arXiv preprint arXiv:2302.00111*, 2023.

[11] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, et al. Scaling rectified flow transformers for high-resolution image synthesis. *arXiv preprint arXiv:2403.03206*, 2024.

[12] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018.

[13] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[14] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.

[15] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention. *arXiv preprint arXiv:2103.03206*, 2021.

[16] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[17] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[18] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. *arXiv preprint arXiv:2506.17811*, 2025.

[19] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[20] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

[21] Dongyang Liu, Shitian Zhao, Le Zhuo, Weifeng Lin, Yi Xin, Xinyue Li, Qi Qin, Yu Qiao, Hongsheng Li, and Peng Gao. Lumina-mGPT: Illuminate flexible photorealistic text-to-image generation with multimodal generative pretraining. *arXiv preprint arXiv:2408.02657*, 2024.

[22] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[23] Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.

[24] Reuven Y. Rubinstein and Dirk P. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.

[25] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[26] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[27] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.